AI with Rav

Learn AI the way you actually think.

DAY 8 of 30

MACHINE LEARNING

Random Forest — the wisdom of the crowd

WHAT YOU'LL LEARN TODAY

- Why one tree alone is risky
- How a "forest" of trees votes together
- The clever trick that makes each tree different
- Why the crowd is almost always right

Random Forest — the wisdom of the crowd

> In One Line: One decision tree can be wrong. So Random Forest asks HUNDREDS of trees the same question and takes a vote. The crowd is almost always right.

Start here: "Ask the Audience"

Remember Kaun Banega Crorepati? You're stuck on a question. You use a lifeline: **Ask the Audience.** One random person in the crowd might be wrong — but when the *whole* audience votes, they nail the right answer almost every time.

Here's the surprising part: **a big crowd of ordinary people beats one lonely expert.** That exact idea, turned into a machine, is called **Random Forest** — and it's one of the most powerful, most-used algorithms in the world.

One tree vs a whole forest

One tree can give the wrong answer. But when a whole forest votes, the majority is almost always right.

Yesterday you met the Decision Tree. It's smart — but it has a weakness: **a single tree can easily be wrong** (remember, it can overfit and memorise).

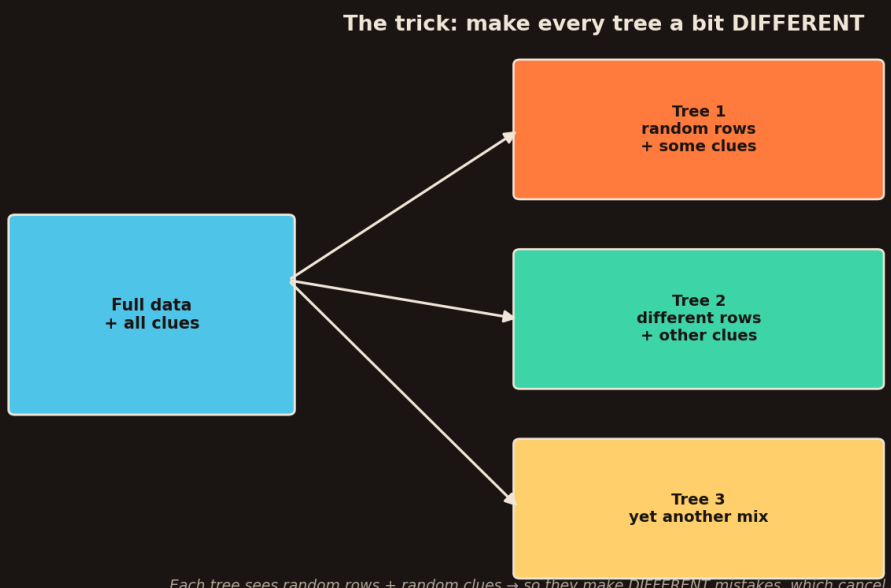
So Random Forest does something simple and brilliant:

- Build **hundreds of trees** instead of one.
- Ask them **all** the same question ("Is this email spam?").
- Take a **vote.** Whatever most trees say → that's the answer.

> One tree says "not spam" and is wrong. But if 87 out of 100 trees say "spam", the forest says **spam** — and it's right. The mistakes of a few trees get out-voted by the many. That's the whole magic.

The clever trick: make every tree different

Wait — if all the trees are the same, they'd all make the *same* mistake, and voting would be pointless. So Random Forest makes sure **every tree is a little bit different.**

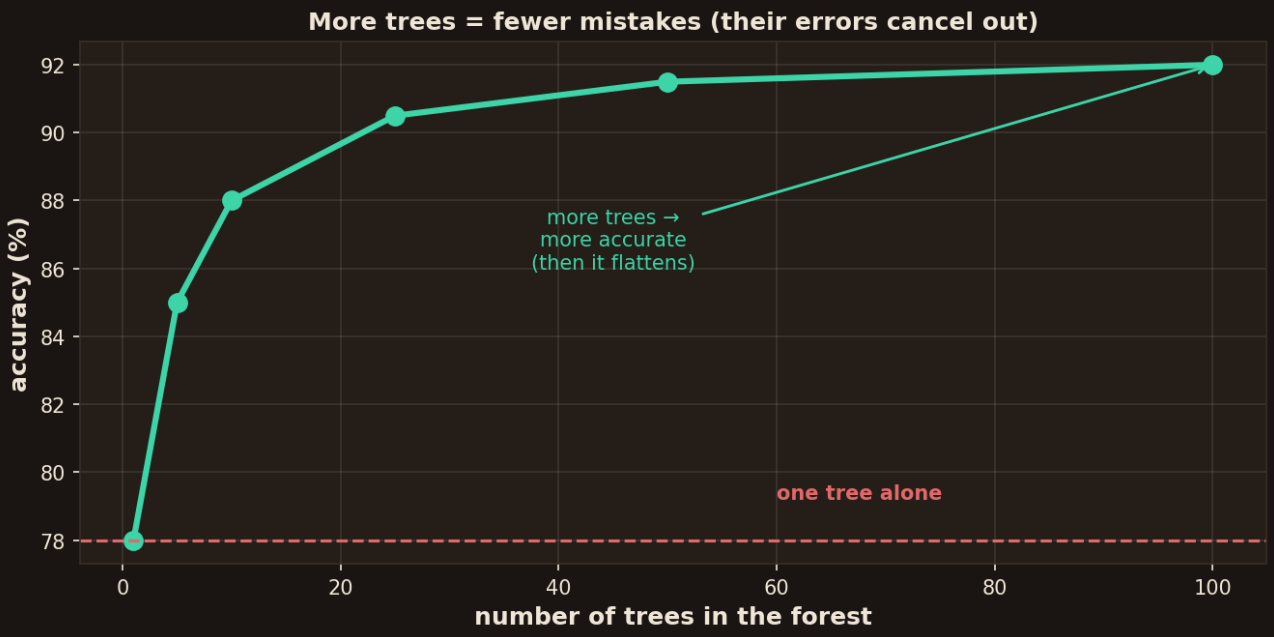


Each tree gets a random handful of rows and a random handful of clues — so no two trees think exactly alike.

- Each tree is trained on a **random sample of the rows** (different data).
- At each question, it can only look at a **random few of the clues** (features), not all of them.
- So every tree grows up seeing the world a little differently — like different people in the audience.

> Because each tree sees different data and different clues, they make **different mistakes.** And here's the beautiful part: when you vote, the random mistakes cancel each other out — while the real pattern, which they all see, stays. That's why the crowd wins.

Why the crowd is almost always right



Add more trees and the forest gets more accurate — then it settles. Many okay trees beat one clever tree.

This "vote and the errors cancel" idea has a name: the tree's mistakes are **random and independent**, so they don't all point the same way — they blur out. The real signal, which every tree picks up, is what survives the vote.

> Simple rule of the crowd: if each tree is right *slightly more often than a coin flip*, then a big group of them voting together becomes **almost always right.** More trees → fewer mistakes (until it flattens out). This is why Random Forest is a favourite for real-world problems — fraud, medical tests, loan approvals.

The formula — but really simple

There's no scary formula here — the "maths" is just **counting votes.** For a yes/no question:

> **Final answer = the option that MORE than half the trees chose (majority vote).**

And if the trees are predicting a **number** (like a house price) instead of yes/no?

- Then each tree gives its own number guess.
- The forest just takes the **average** of all their guesses.

> That's it. Classification → **majority vote.** Numbers → **average.** No calculus, no derivatives. The power comes from *many* trees, not fancy maths.

Train one — a few lines

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)   # 100 trees
model.fit(X_train, y_train)

print(model.predict([[income, age, has_credit]]))   # the forest's vote
print(model.feature_importances_)   # which clues mattered most!
```

> `n_estimators=100` means "grow 100 trees." Bonus: `feature_importances_` tells you which clues the forest relied on most — great for explaining your model to your boss.

When it's not the best choice

- **Slower and bigger** → 100 trees take more time and memory than one. For a tiny quick task, one tree may be enough.
- **Harder to read** → you *can* read one tree's rules, but reading 100 is not practical. You trade a bit of "explainability" for a lot of accuracy.
- **Not for everything** → for images, text, and speech, deep learning (coming later) usually wins. Random Forest shines on **table data** (rows and columns) — which is most business data.

Recap — the 20-second version

- Random Forest = **ask many trees and vote** — the wisdom of the crowd.
- One tree can be wrong; a **forest out-votes** the mistakes.
- The trick: each tree sees **random rows + random clues**, so they differ.
- Yes/no → **majority vote.** Numbers → **average.** No scary maths.
- Best on **table data** (fraud, loans, medical) — a real-world workhorse.

> Next up — Video 9: XGBoost, the "Kaggle king". Instead of trees voting all at once, what if each new tree learns from the mistakes of the ones before it? Learning from your own errors, round after round. See you Day 9.